

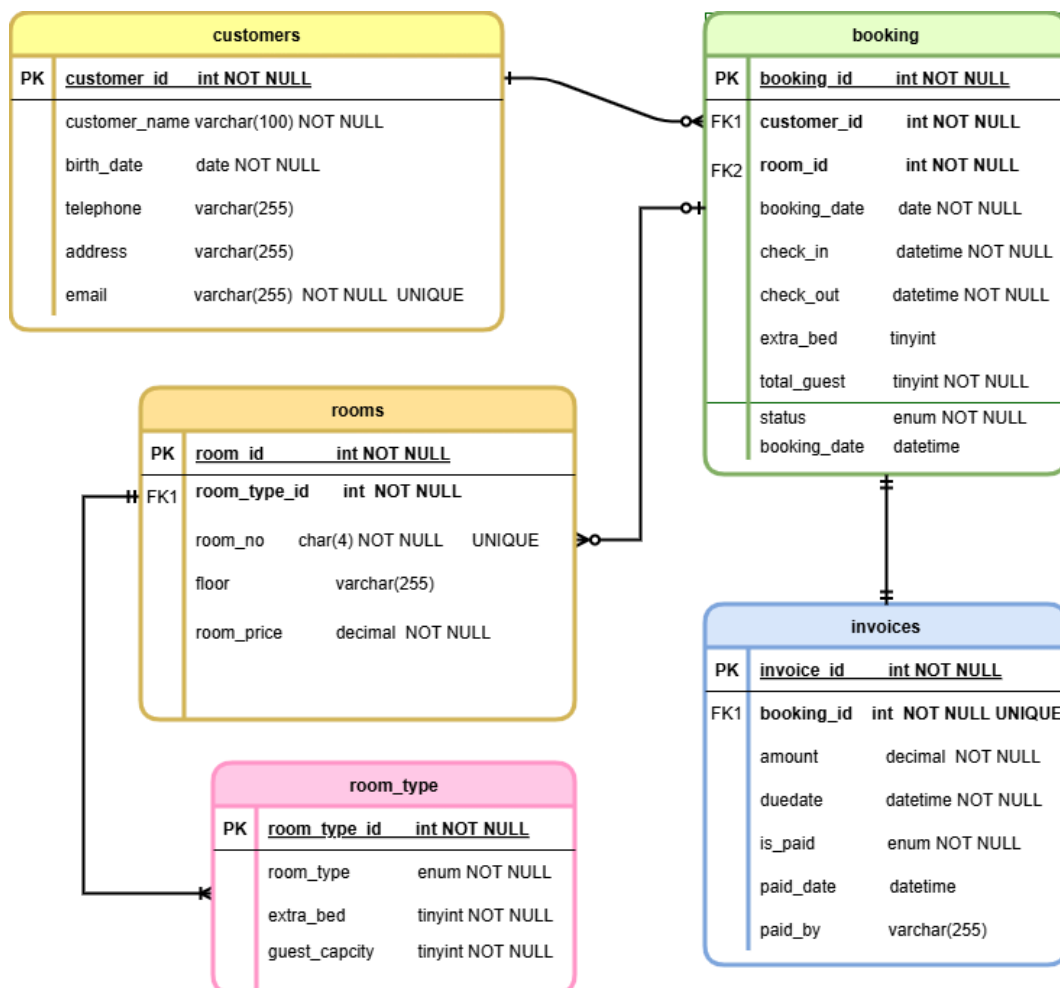
Inlämningsuppgift 2

Database Design and Explanation (Hotel Booking System)

1. Introduction

This project is a hotel booking system. The system allows customers to register, book hotel rooms, and pay invoices. It also allows administrators to manage customers, bookings, and view statistics. This assignment is a continuation of **Inlämningsuppgift 1**, where a first ERD was created. In **Inlämningsuppgift 2**, the ERD has been updated to match the final database design that is actually used in the application.

The final ERD contains five tables: **customers**, **rooms**, **room_type**, **booking**, and **invoices**. This report explains the ERD table by table, describes important constraints, compares the first and final designs, and presents a NoSQL design for hotel activities.



2. Final ERD – Tables and Relationships

Customers:

The **customers** table stores information about hotel customers. It contains customer id (which is a GUID instead of a running number), customer name, birth date, telephone number, address, and email.

The **customer id** is stored as a **GUID** to make sure each customer has a unique id and to avoid duplicate ids.

The **birth_date** column is **NOT NULL**, because it is used to check that customers under 18 years old are not allowed to book a room.

The **email** column is **NOT NULL** and **UNIQUE**, because email is used for login and each customer should only have one account.

One customer can make many bookings. Therefore, the **customers** table has a **one-to-many relationship** with the **booking** table.

Rooms:

The **rooms** table stores information about each physical room in the hotel. It contains room id, room number, floor, room price, and room type.

The **room number** is **NOT NULL** and **UNIQUE**, which means each room can be clearly identified.

The **room price** is **NOT NULL**, because a room must always have a price.

Each room belongs to one room type, but the same room can be booked many times on different dates. Therefore, the **rooms** table has a **one-to-many relationship** with the **booking** table.

Room_Type:

The **room_type** table stores information about different types of rooms, such as single and double rooms. It contains room type, guest capacity, and number of extra beds.

All columns are **NOT NULL**, because this information is needed to check if a booking is valid.

The **room_type** column is stored as an **enum**, so only allowed values can be used.

The **extra_bed** column in the **room_type** table shows the maximum number of extra beds that are allowed for that room type. For example, a double room may allow one extra bed, while a single room may not allow any.

One room type can be connected to many rooms. This creates a **one-to-many relationship** between **room_type** and **rooms**.

Booking:

The **booking** table stores information about hotel bookings and connects customers with rooms. It contains booking date, check-in date, check-out date, total number of guests, extra beds, and booking status.

The **booking status** is stored as an enum and can be **ACTIVE**, **CANCELLED**, or **EXPIRED**. This makes it easier to control the booking process.

Each booking belongs to one customer and one room. The system prevents overlapping bookings for the same room using application logic.

The **extra_bed** column in the **booking** table shows how many extra beds are actually used in a specific booking. This value is checked against the **room_type.extra_bed** limit, so the system does not allow more extra beds than the room type supports. In summary, **room_type.extra_bed** defines the allowed limit, while **booking.extra_bed** stores the actual number used by the customer. Together with **guest_capacity**, this ensures that the total number of guests does not exceed what the room can handle.

Booking is also used to control payment time. If the invoice is not paid within **10 days**, the booking is automatically cancelled.

Invoices:

The **invoices** table stores payment information for bookings. It contains invoice id, booking id, amount, due date, payment status, paid date, and paid by.

Each booking can only have **one invoice**, which creates a **one-to-one relationship** between **booking** and **invoices**. Each booking can only have one invoice. This means the same **booking_id** cannot be used more than once in the invoices table.

Payment status is stored as an enum with the values PAID and UNPAID, so only allowed values can be used.

3. Data Types and Constraints

Different data types are used depending on the type of data:

- **INT** for id values
- **VARCHAR** for text such as names, addresses, and emails
- **DATE / DATETIME** for booking and payment dates
- **DECIMAL** for prices and amounts
- **ENUM** for status fields

Important constraints include:

- **NOT NULL** for required fields

- **UNIQUE** for email and room number
- **FOREIGN KEY** to connect related tables
- **ENUM** to restrict allowed values

These constraints ensure data integrity and prevent invalid data from being stored.

4. Reflection between Inlämningsuppgift 1 and Inlämningsuppgift 2

In Inlämningsuppgift 1, the ERD was a draft design. The main goal was to understand the basic structure of the hotel booking system and how the tables are connected.

In Inlämningsuppgift 2, the ERD was updated to match the database that is actually implemented in the application.

One important improvement is the introduction of the `room_type` table. In the first version, room type information was stored directly in the `rooms` table. By moving it into a separate table, the database became more organized and easier to maintain. More constraints were also added in the final design, such as **UNIQUE** constraints for email, room number, and `booking_id` in invoices. These changes were made after implementing the system and seeing where data control is important.

The final ERD matches the Python code and application logic. Rules such as preventing overlapping bookings and automatic cancellation after 10 days are handled in the application, which influenced the final database design. Overall, Inlämningsuppgift 2 presents a clearer and more reliable database design than the first version.

5. NoSQL Design for Hotel Activities

In addition to hotel room bookings, the hotel offers activities such as **spa treatments** and **kayaking tours**. These activities have flexible structures and different attributes, which makes them suitable for a NoSQL database. A NoSQL database allows activity data to change without affecting the relational database.

Example 1: Spa Activity (NoSQL Document)

```
spa = {
  "activity_id": "A_001",
  "name": "Spa Relax Package",
  "type": ["oil massage", "foot massage", "swedish massage", "thai massage"],
  "duration_minutes": [30, 60, 90, 120],
  "price": [450, 800, 1200, 1500],
  "therapist": "Mia",
  "available_time": "14:00",
```

```
"active": True  
}
```

Example 2: Kayaking Tour (NoSQL Document)

```
kayaking = {  
  
  "activity_id": "A_002",  
  
  "name": "Kayaking Tour",  
  
  "type": "outdoor",  
  
  "price": 800,  
  
  "capacity": 10,  
  
  "location": ["Lake ", "Waterfall"]  
  
  "requirements": ["Can swim", "Age 12+"],  
  
  "active": true  
}
```

According to these examples, NoSQL is suitable for activities because different activities have different attributes. The structure can change over time depending on the hotel, and new activity types can be added easily. Activities are also not tightly connected to the booking system, which makes this solution safe for the main system.

6. Conclusion

In conclusion, the final ERD represents the database structure used in the hotel booking application. The relational database keeps customer, room, booking, and invoice data consistent and correct. NoSQL is used for activities to provide flexibility and allow future growth. I think this design is simple, easy to understand, and suitable for a basic hotel booking system. It also makes it possible to add more features in the future.